Imports and Loading the Data.

In [4]:
```
import pandas as pd
import numpy as np
import os
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

# in Jupyter notebooks __file__ is not defined, use current working director
BASE_DIR = os.getcwd()
df = pd.read_csv(os.path.join(BASE_DIR, '../dataset/allcases.csv'))
```

The Cleanup and Splitting

In [5]:
```
#  Drop the "ghost" Excel columns (anything starting with 'Unnamed')
df = df.loc[:, ~df.columns.str.contains('^Unnamed')]

#  Drop columns missing more than 50% of their data
# thresh requires a minimum number of NON-NA values to keep the column
threshold = len(df) * 0.5
df = df.dropna(thresh=threshold, axis=1)

# Save all our potential targets separately
y_diagnosis = df['Diagnosis']
y_management = df['Management']
y_severity = df['Severity']

# Make sure NO targets are in our features (X)
# We also drop 'Diagnosis_Presumptive' as it's a doctor's guess before the f
targets_to_drop = ['Diagnosis', 'Management', 'Severity', 'Diagnosis_Presump
X = df.drop(columns=targets_to_drop)

# Impute missing values (from the step we just did)
numeric_cols = X.select_dtypes(include=['number']).columns
X[numeric_cols] = X[numeric_cols].fillna(X[numeric_cols].median())

categorical_cols = X.select_dtypes(include=['object']).columns
for col in categorical_cols:
    X[col] = X[col].fillna(X[col].mode()[0])

# Encode Categorical Variables (One-Hot Encoding)
# This turns text columns into multiple binary (0 or 1) columns
X_encoded = pd.get_dummies(X, columns=categorical_cols, drop_first=True)

# Scale the Data
# Logistic Regression needs all numbers to be on a similar scale
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X_encoded)

# Convert back to a DataFrame just so we can see the column names if we want
X_final = pd.DataFrame(X_scaled, columns=X_encoded.columns)

# Split the Data! (80% for training, 20% held out for final testing)
```

```
# We will use y_diagnosis as our target for this first Logistic Regression m
X_train, X_test, y_train, y_test = train_test_split(X_final, y_diagnosis, te

print("\n--- Final Data Prep ---")
print(f"Training features shape: {X_train.shape}")
print(f"Testing features shape: {X_test.shape}")
print("Data is scaled, encoded, and ready for TensorFlow!")
```

```
--- Final Data Prep ---
Training features shape: (625, 45)
Testing features shape: (157, 45)
Data is scaled, encoded, and ready for TensorFlow!
```

```
/var/folders/jw/xfwp5tfs0wqc3vnyzr3wn_c80000gn/T/ipykernel_26625/197474058
6.py:23: Pandas4Warning: For backward compatibility, 'str' dtypes are includ
ed by select_dtypes when 'object' dtype is specified. This behavior is depre
cated and will be removed in a future version. Explicitly pass 'str' to `inc
lude` to select them, or to `exclude` to remove them and silence this warnin
g.
See https://pandas.pydata.org/docs/user_guide/migration-3-strings.html#strin
g-migration-select-dtypes for details on how to write code that works with p
andas 2 and 3.
  categorical_cols = X.select_dtypes(include=['object']).columns
```

The TensorFlow Mode

In [6]:

```
import tensorflow as tf
import keras
from keras.models import Sequential
from keras.layers import Dense, Input
from sklearn.preprocessing import LabelEncoder
from sklearn.model_selection import StratifiedKFold
import numpy as np

def build_logistic_model(input_dim):
    model = Sequential([
        Input(shape=(input_dim,)),
        Dense(1, activation='sigmoid')
    ])
    model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['ac
    return model

def run_kfold_logistic(X_tr, y_tr, X_te, y_te, label, n_splits=5):
    """Run StratifiedKFold CV then evaluate on holdout test set."""
    skf = StratifiedKFold(n_splits=n_splits, shuffle=True, random_state=42)
    fold_accs = []

    print(f"\n--- {label} | Logistic Regression | {n_splits}-Fold CV ---")
    for fold, (train_idx, val_idx) in enumerate(skf.split(X_tr, y_tr)):
        X_f_tr, X_f_val = X_tr[train_idx], X_tr[val_idx]
        y_f_tr, y_f_val = y_tr[train_idx], y_tr[val_idx]

        m = build_logistic_model(X_tr.shape[1])
        m.fit(X_f_tr, y_f_tr, epochs=50, batch_size=32, verbose=0)
        _, acc = m.evaluate(X_f_val, y_f_val, verbose=0)
        fold_accs.append(acc)
        print(f"  Fold {fold+1} Val Accuracy: {acc*100:.2f}%")
```

```
    print(f"Mean CV Accuracy: {np.mean(fold_accs)*100:.2f}% (+/- {np.std(fol

    # Final model trained on all training data, evaluated on holdout
    final_model = build_logistic_model(X_tr.shape[1])
    final_model.fit(X_tr, y_tr, epochs=50, batch_size=32, verbose=0)
    _, test_acc = final_model.evaluate(X_te, y_te, verbose=0)
    print(f"Holdout Test Accuracy: {test_acc*100:.2f}%")
    return final_model

# ── DIAGNOSIS (binary) ──────────────────────────────────────────────
# Drop NaN rows in y_diagnosis
valid_train = y_train.dropna().index
valid_test = y_test.dropna().index

le_diag = LabelEncoder()
y_tr_diag = le_diag.fit_transform(y_train.loc[valid_train])
y_te_diag = le_diag.transform(y_test.loc[valid_test])
X_tr_diag = X_train.loc[valid_train].values
X_te_diag = X_test.loc[valid_test].values

print("Diagnosis classes:", le_diag.classes_)
model_diag = run_kfold_logistic(X_tr_diag, y_tr_diag, X_te_diag, y_te_diag,
```

```
2026-04-08 18:21:01.544544: I metal_plugin/src/device/metal_device.cc:1154]
Metal device set to: Apple M2 Max
2026-04-08 18:21:01.544578: I metal_plugin/src/device/metal_device.cc:296] s
ystemMemory: 96.00 GB
2026-04-08 18:21:01.544582: I metal_plugin/src/device/metal_device.cc:313] m
axCacheSize: 38.88 GB
2026-04-08 18:21:01.544754: I tensorflow/core/common_runtime/pluggable_devic
e/pluggable_device_factory.cc:305] Could not identify NUMA node of platform
GPU ID 0, defaulting to 0. Your kernel may not have been built with NUMA sup
port.
2026-04-08 18:21:01.544765: I tensorflow/core/common_runtime/pluggable_devic
e/pluggable_device_factory.cc:271] Created TensorFlow device (/job:localhos
t/replica:0/task:0/device:GPU:0 with 0 MB memory) -> physical PluggableDevic
e (device: 0, name: METAL, pci bus id: <undefined>)
Diagnosis classes: ['appendicitis' 'no appendicitis']

--- Diagnosis | Logistic Regression | 5-Fold CV ---
2026-04-08 18:21:01.788053: I tensorflow/core/grappler/optimizers/custom_gra
ph_optimizer_registry.cc:117] Plugin optimizer for device_type GPU is enable
d.
  Fold 1 Val Accuracy: 81.60%
  Fold 2 Val Accuracy: 83.20%
  Fold 3 Val Accuracy: 85.60%
  Fold 4 Val Accuracy: 81.45%
  Fold 5 Val Accuracy: 83.87%
Mean CV Accuracy: 83.14% (+/- 1.54%)
Holdout Test Accuracy: 80.25%
```

Severity Target — Logistic Regression with K-Fold CV

In [7]:
```
# ── SEVERITY (binary: uncomplicated / complicated) ──────────────────
from sklearn.utils.class_weight import compute_class_weight
```

```
from sklearn.metrics import confusion_matrix, classification_report

# Align y_severity to the same train/test split indices used for X
y_sev_train = y_severity.loc[X_train.index]
y_sev_test = y_severity.loc[X_test.index]

valid_train_sev = y_sev_train.dropna().index
valid_test_sev = y_sev_test.dropna().index

le_sev = LabelEncoder()
y_tr_sev = le_sev.fit_transform(y_sev_train.loc[valid_train_sev])
y_te_sev = le_sev.transform(y_sev_test.loc[valid_test_sev])
X_tr_sev = X_train.loc[valid_train_sev].values
X_te_sev = X_test.loc[valid_test_sev].values

print("Severity classes:", le_sev.classes_)
print("Severity distribution (train):", dict(zip(*np.unique(y_tr_sev, return

# Compute class weights to counter imbalance (85% uncomplicated vs 15% compl
weights = compute_class_weight('balanced', classes=np.unique(y_tr_sev), y=y_
class_weight_dict = dict(enumerate(weights))
print("Class weights:", class_weight_dict)

# K-Fold with class weights
skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
fold_accs = []

print("\n--- Severity | Logistic Regression | 5-Fold CV (with class weights)
for fold, (train_idx, val_idx) in enumerate(skf.split(X_tr_sev, y_tr_sev)):
    X_f_tr, X_f_val = X_tr_sev[train_idx], X_tr_sev[val_idx]
    y_f_tr, y_f_val = y_tr_sev[train_idx], y_tr_sev[val_idx]

    m = build_logistic_model(X_tr_sev.shape[1])
    m.fit(X_f_tr, y_f_tr, epochs=50, batch_size=32, verbose=0, class_weight=
    _, acc = m.evaluate(X_f_val, y_f_val, verbose=0)
    fold_accs.append(acc)
    print(f"  Fold {fold+1} Val Accuracy: {acc*100:.2f}%")

print(f"Mean CV Accuracy: {np.mean(fold_accs)*100:.2f}% (+/- {np.std(fold_ac

# Final model on all training data
model_sev = build_logistic_model(X_tr_sev.shape[1])
model_sev.fit(X_tr_sev, y_tr_sev, epochs=50, batch_size=32, verbose=0, class

_, test_acc = model_sev.evaluate(X_te_sev, y_te_sev, verbose=0)
print(f"Holdout Test Accuracy: {test_acc*100:.2f}%")

# Confusion matrix — shows if model actually catches "complicated" cases
y_pred_sev = (model_sev.predict(X_te_sev) > 0.5).astype(int).flatten()
print("\nConfusion Matrix:")
print(confusion_matrix(y_te_sev, y_pred_sev))
print("\nClassification Report:")
print(classification_report(y_te_sev, y_pred_sev, target_names=le_sev.classe
```

```
Severity classes: ['complicated' 'uncomplicated']
Severity distribution (train): {0: 95, 1: 529}
Class weights: {0: 3.2842105263157895, 1: 0.5897920604914934}

--- Severity | Logistic Regression | 5-Fold CV (with class weights) ---
  Fold 1 Val Accuracy: 81.60%
  Fold 2 Val Accuracy: 74.40%
  Fold 3 Val Accuracy: 75.20%
  Fold 4 Val Accuracy: 80.80%
  Fold 5 Val Accuracy: 79.84%
Mean CV Accuracy: 78.37% (+/- 2.98%)
Holdout Test Accuracy: 80.25%
5/5 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step

Confusion Matrix:
[[ 22   2]
 [ 29 104]]

Classification Report:
               precision    recall  f1-score   support

  complicated       0.43      0.92      0.59        24
uncomplicated       0.98      0.78      0.87       133

     accuracy                           0.80       157
    macro avg       0.71      0.85      0.73       157
 weighted avg       0.90      0.80      0.83       157
```

Management Target — Multiclass Logistic Regression with K-Fold CV

In [8]:

```
# ── MANAGEMENT (3-class: conservative / primary surgical / secondary surgic
from keras.utils import to_categorical

# Align y_management to the same train/test split indices
y_mgmt_train = y_management.loc[X_train.index]
y_mgmt_test  = y_management.loc[X_test.index]

# Merge the 1-sample "simultaneous appendectomy" into "secondary surgical"
y_mgmt_train = y_mgmt_train.replace('simultaneous appendectomy', 'secondary
y_mgmt_test  = y_mgmt_test.replace('simultaneous appendectomy', 'secondary s

valid_train_mgmt = y_mgmt_train.dropna().index
valid_test_mgmt  = y_mgmt_test.dropna().index

le_mgmt = LabelEncoder()
y_tr_mgmt = le_mgmt.fit_transform(y_mgmt_train.loc[valid_train_mgmt])
y_te_mgmt = le_mgmt.transform(y_mgmt_test.loc[valid_test_mgmt])
X_tr_mgmt = X_train.loc[valid_train_mgmt].values
X_te_mgmt = X_test.loc[valid_test_mgmt].values

n_classes = len(le_mgmt.classes_)
print("Management classes:", le_mgmt.classes_)
print("Distribution (train):", dict(zip(*np.unique(y_tr_mgmt, return_counts=

# One-hot encode labels for categorical_crossentropy
```

```
y_tr_mgmt_oh = to_categorical(y_tr_mgmt, num_classes=n_classes)
y_te_mgmt_oh = to_categorical(y_te_mgmt, num_classes=n_classes)

# Class weights to handle imbalance (conservative >> primary surgical >> sec
weights_mgmt = compute_class_weight('balanced', classes=np.unique(y_tr_mgmt)
class_weight_mgmt = dict(enumerate(weights_mgmt))
print("Class weights:", class_weight_mgmt)

def build_multiclass_model(input_dim, n_classes):
    model = Sequential([
        Input(shape=(input_dim,)),
        Dense(n_classes, activation='softmax')
    ])
    model.compile(optimizer='adam', loss='categorical_crossentropy', metrics
    return model

# K-Fold CV
skf_mgmt = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
fold_accs_mgmt = []

print("\n--- Management | Logistic Regression | 5-Fold CV (with class weight
for fold, (train_idx, val_idx) in enumerate(skf_mgmt.split(X_tr_mgmt, y_tr_m
    X_f_tr, X_f_val = X_tr_mgmt[train_idx], X_tr_mgmt[val_idx]
    y_f_tr = to_categorical(y_tr_mgmt[train_idx], num_classes=n_classes)
    y_f_val = to_categorical(y_tr_mgmt[val_idx], num_classes=n_classes)

    m = build_multiclass_model(X_tr_mgmt.shape[1], n_classes)
    m.fit(X_f_tr, y_f_tr, epochs=50, batch_size=32, verbose=0, class_weight=
    _, acc = m.evaluate(X_f_val, y_f_val, verbose=0)
    fold_accs_mgmt.append(acc)
    print(f"  Fold {fold+1} Val Accuracy: {acc*100:.2f}%")

print(f"Mean CV Accuracy: {np.mean(fold_accs_mgmt)*100:.2f}% (+/- {np.std(fo

# Final model on all training data
model_mgmt = build_multiclass_model(X_tr_mgmt.shape[1], n_classes)
model_mgmt.fit(X_tr_mgmt, y_tr_mgmt_oh, epochs=50, batch_size=32, verbose=0,

_, test_acc_mgmt = model_mgmt.evaluate(X_te_mgmt, y_te_mgmt_oh, verbose=0)
print(f"Holdout Test Accuracy: {test_acc_mgmt*100:.2f}%")

# Confusion matrix & classification report
y_pred_mgmt = np.argmax(model_mgmt.predict(X_te_mgmt), axis=1)
print("\nConfusion Matrix:")
print(confusion_matrix(y_te_mgmt, y_pred_mgmt))
print("\nClassification Report:")
print(classification_report(y_te_mgmt, y_pred_mgmt, target_names=le_mgmt.cla
```

```
Management classes: ['conservative' 'primary surgical' 'secondary surgical']
Distribution (train): {0: 386, 1: 215, 2: 23}
Class weights: {0: 0.538860103626943, 1: 0.9674418604651163, 2: 9.0434782608
69565}

--- Management | Logistic Regression | 5-Fold CV (with class weights) ---
  Fold 1 Val Accuracy: 74.40%
  Fold 2 Val Accuracy: 76.80%
  Fold 3 Val Accuracy: 70.40%
  Fold 4 Val Accuracy: 75.20%
  Fold 5 Val Accuracy: 73.39%
Mean CV Accuracy: 74.04% (+/- 2.13%)
Holdout Test Accuracy: 80.25%
5/5 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step

Confusion Matrix:
[[79  7 11]
 [ 4 43  8]
 [ 0  1  4]]

Classification Report:
                    precision    recall  f1-score   support

      conservative       0.95      0.81      0.88        97
  primary surgical       0.84      0.78      0.81        55
secondary surgical       0.17      0.80      0.29         5

          accuracy                           0.80       157
         macro avg       0.66      0.80      0.66       157
      weighted avg       0.89      0.80      0.84       157
```